Node JS

Day-0

4 Feb, 2026

Node JS

- Node JS is a free, open source and cross platform Javascript runtime, used by developers to build web applications, servers, command line tools and scripts.

- Web Application runs on internet by following the standards and principles of Web.
- Server is responsible for hosting applications.
- Command line tools can allow to control any application with commands in a CLI. [Command Line Interface]
- Scripts are hosted on server to generate a customized response for every client request.

- There are various server side scripting technologies like **CGI, ASP, JSP, PHP, Node JS** etc.

Features of Node JS:

1. It is single threaded.

Node JS

Day-1

•

Feb 6, 2026

Summary:

- What is server side scripting?
- What is Node JS?

Features of Node JS:

1. Single Threaded

2. Asynchronous

    - It uses unblocking technique.

    - It improves the performance of application.

3. Modular

    - Application specific framework.

    - It makes application light weight.

    - Easy reusability, extensibility, maintainability & testability.

4. Event Driven Architecture

    - Easy to configure action triggers

    - Event Loop

    - Event Listener

5. No Buffering

    - Data is received in the form of memory packet.

    - It will not buffer the data.

6. It uses Javascript as language.

7. It is open source and cross platform.

8. Huge set of packages.

9. Large Community

10. Can plugin in with various front-end technologies easily.

Node JS Objects & Techniques

- Node JS can deploy scripts on server, in order to generate a response customized for

every client request.

- It uses Javascript as language.
- It comprises of a set of objects like
    a) Request
    b) Response
    c) Session
    d) Application etc.
- It handles various techniques like
    a) Model Binding
    b) Data Binding
    c) Event Binding
    d) Routing
    e) Caching etc.

Note: Server side scripting handles interactions using objects & techniques.

Javascript Server Side:

- Node JS uses Javascript as a language for server side interactions.
- JavaScript server side is used only as language not for manipulating DOM & BOM.
- Javascript language server side incudes

1. Variables
2. Data Types
3. Operators
4. Statements
5. Functions

6. OOP

- Modules
- Classes
- Inheritance
- Polymorphism

Setup Environment for Server Side Node JS:

1. Install Node JS

2. Create a new folder for server app

d:\server-app

3. Setup package json

> npm init -y

4. Create index.js or main.js as startup for project

5. Enable ESLint for project, which is Javascript language analysis tool.

> npm init @eslint/config@latest

What do you want to lint? · javascript

How would you like to use ESLint? · problems

What type of modules does your project use? · script

Which framework does your project use? · none

Does your project use TypeScript? · No / Yes

Where does your code run? ◉ Node

6. Server side Javascript files are compiled by using a Node compiler

```
> node  index.js
```

FAQ: What is Node REPL?

Ans:

- Node JS provides a terminal.
- Terminal is a command line tool used by developers to test node actions like

```
R        - Read
E        - Execute
P        - Print
L        - Loop
```

- You can invoke the terminal by using

```
C:\> node
>
```

- Press Ctrl + C twice to exit node terminal

Node Modules

- Node JS provides a set of modules that are required to build a server side application.
- Technically module is a set of
  a) Classes
  b) Functions
  c) Values
- Node provides several pre-defined modules to build server side interactions.
- It also allows to create custom modules.
- Modules allow
  a) Reusability
  b) Extensibility
  c) Maintainability
  d) Testability

Node built-in Modules:

1. OS
   - It provides a set of properties and methods that are used to get server OS details.
   - OS details include CPU structure, Free memory, Server Name, Platform etc.

Syntax:

```
const os = require("os");
console.log(os.cpus());
```

console.log(os.freemem());

- Properties and method shown below

arch

availableParallelism

cpus

endianness

freemem

getPriority

homedir

hostname etc.

Node path and fs module

Day-2

Feb 7, 2026

Node Modules

1. **OS**

2. **Path**

   - It provides classes with properties and methods that are used to access the server location details.
   - It also provides members to get information about any specific file or folder.
   - Path details include

     a) dirname

     b) extname

     c) isAbsolute

d) format [MIME]

e) delimiter etc.

Syntax:

```
const  path = require("path");
path.extname( "file_path_and_name" );
path.dirname( "file_path");
```

Ex: index.js

```
const path = require("path");
var ext = path.extname('/project/src/photo.jpg');
if(ext===='.jpg'){
  console.log('Uploaded successfully..');
} else {
  console.log('You can upload only JPG files');
}
```

3. FS

- It is a file stream module.
- It provides a set of properties and methods defined with various classes, which are required to handle files and folders on server.

Reading File Content:

a) Synchronous Technique

- It uses a **blocking** technique.
- It blocks all other tasks in the request while executing the specified.
- It moves to next only when it completes the current.
- **FS** module provides **“readFileSync()”** method to read the content using sync technique.

Syntax:

```
const fs = require(“fs”);

let data = fs.readFileSync( “filename” );
console.log(data.toString());
```

Ex:

```
const fs = require("fs");

console.log('Before Reading File Content');

let data = fs.readFileSync("./readme.txt");
console.log(data.toString());

console.log('After reading file content');
```

b) Asynchronous Techniques

- It is an unblocking technique.
- It can execute the given task without blocking the other tasks in process.
- It can handle multiple tasks simultaneously at the same time.

- You can read using unblocking technique with **“readFile()”** method.

Syntax:

```
fs.readFile( “path”, function( error, data ){


})
```

- Error object comprises of various error details like

  i) name

  ii) message

  iii) code

  iv) path etc.

- Data comprises of the data returned by file in **binary** format.

Ex: index.js

```
const fs = require("fs");


console.log('Before Reading File Content');


fs.readFile("./readme.txt", (error, data)=>{

    if(!error){

      console.log(data.toString());

    } else {

      console.log(error);

    }
```

```
})

console.log('After reading file content');
```

Writing Content into File:

- File stream object provides both async and sync methods for writing data into file.
- To replace the existing content with new content you can use

  a) writeFile() : Unblocking

  b) writeFileSync() : Blocking

Syntax:

```
fs.writeFile( “filePath”, “data”, function(error) {

})
```

- To add data into file without replacing the existing content you can use

  a) appendFile() : Unblocking

  b) appendFileSync() : Blocking

Syntax:

```
fs.appendFile( “filePath”, “\n new data”, function(error) {

})
```

EX:

```
const fs = require("fs");
```

```
fs.appendFile('./readme.txt', '\nLine 2',(err)=>{
    if(!err){
        console.log('Data Written to file successfully..');
    } else {
        console.log(err);
    }
})
```

Removing File

- You can remove file from server location by using “unlink()” method.
- You can use blocking technique with “unlinkSync()”

Syntax:

```
fs.unlink(“filePath”, (error) => { })
```

Ex:

```
const fs = require("fs");

fs.unlink('./readme.txt',(error)=>{
   if(!error){
      console.log('Deleted..');
   }
})
```

Http Module

Day-3

Feb 9, 2026

Summary - Node Modules

- OS
- FS
- PATH

URL Module

- It provides the properties and methods required to access the URL.
- It contains complete information about the virtual path.
- URL is a virtual path used to locate the resources on server.
- It is also used in parsing URL.
- It provides various classes like
    a) URL
    b) URLSearchParams etc.
- You can configure an object to access and use members of class.

Syntax:

```
var params = new URLSearchParams('url');
params.get(key);
```

Events Module

- Events are derived from **EventEmitter** base class.

- You can configure a custom event by using an object of **EventEmitter**.

Syntax:

```
const  events = require("events");

const  eventEmitter = new events.EventEmitter();

const  emitter = new eventEmitter();

emitter.on( "eventName", function( args ) { });

emitter.emit("eventName");
```

- All Javascript event basics are same

1. What is Event
2. What is Event Handler
3. What is Event Listener
4. Event Arguments
5. Event Propagation
6. Event Prevent Default
7. Event Loop

Ex:

```
const eventEmitter = require("events").EventEmitter;
```

```
let emitter = new eventEmitter();


emitter.on('Pay', ()=>{
        console.log('Payment Initiated');
})
emitter.emit('Pay');
```

- Event can have arguments, which can any of the primitive and non-primitive types.

Syntax:

```
emitter.on (‘event’, (args) => { } )
```

- You can overload the event with various functionalities, so that it execute the required

  according to the state and situation.

Syntax:

```
emitter.on( ‘event’,  ( ) => { action 1 } )
emitter.on( ‘event’,  ( ) => { action 2 } )


emitter.emit(‘event’);     // executes all listeners. [actions]
```

Ex:

```
const eventEmitter = require("events").EventEmitter;

let emitter = new eventEmitter();

emitter.on('Pay', (bank, user)=>{
      console.log(`Payment Initiated to ${user} of ${bank} bank`);
});
emitter.on('Pay', (bank)=>{
     console.log(`Payment Processed and wating for confirmation from ${bank} bank`);
});
emitter.on('Pay', ()=>{
     console.log('Payment not initiated please provide bank and user details');
});

emitter.emit('Pay', 'AXIS', 'JOHN');
```

- Event is subscribed using “on()” and unsubscribed using “off()”.

Syntax:

```
emitter.off('event');        // turn off event and clean up the memory.
```

- Event uses Synchronous technique to execute the actions.
- You can use event loop which can execute actions according to priority

a) normal task

b) micro task [promise]

c) macro [setTimeout]

Syntax:

```
emitter.on("event", ()=>{
    // normal
    // micro
    // macro
})
```

HTTP Module

- It is used to create a web server or HTTP server.
- HttpServer manages request from client and sends a customized response.

Creating Web Server:

1. Import http module

```
const http = require("http");
```

2. Create a server

```
const server = http.createServer( function(request, response) {

})
```

- Request object provides the properties and methods to fetch data from client.
- It can be
    a) query string
    b) form data
    c) cookies etc.
- Response object provides members to send response, which includes
    a) file
    b) html
    c) xml

d) json etc.

3. Start server

server.listen(port);

Ex:

```
const http = require("http");

const server = http.createServer((req, res)=>{
    res.end('Welcome to Node Web Server');
});

server.listen(2020);
console.log('Server Started : http://127.0.0.1:2020&#39;);
```

FAQ's:

1. What is Web Server?

A. - Web server resembles both hardware and software. - It is responsible for handling request from client and set a customized response for every client request.

- It follows the rules of **HTTP. [**Hyper Text Transfer Protocol]

2. What is a Protocol?

A. Protocol defines a set of rules that specify how devices in a network communicate

with each other. There are various protocols for various network systems

a) HTTP - for Web but unsecured

b) HTTPS - for secured web. [Hyper Text Transfer Protocol Secured]

c) FTP - for file system. [File Transfer Protocol]

d) SMTP - for mail system. [Simple Mail Transfer Protocol]

e) MSMQ - for messaging system. [Message Service Message Queue].

etc.

Server Request and Response

Day-4

Feb 10, 2026

Summary

- **What is Http Module**
- **What is Protocol**
- **How to create server**

FAQ: What is the range of port numbers?

Ans:

0 to 1023 Well Known Port Numbers

1024 to 49151 Registered Port Numbers

49152 to 65535 Dynamic Port Numbers

FAQ: What is Port number?

Ans: It refers to a location where the application is hosted on server.

Every server has IP address and various port numbers to handle different apps.

80	Well known for HTTP
25	Well known for SMTP
8080	Alternative for HTTP
27017	Registered for MongoDB
50000	API / Temporary App services etc.

FAQ: What is difference between URL & URI ?

Ans: URL is **Uniform Resource Locator**

http://server.com/key=value

URI is **Uniform Resource Identifier**

?key=value [Key is ID]

URN is **Uniform Resource Name**

?name=value [Name is reference name]

FAQ: What is the role of Request Object on server?

Ans: It is responsible for requesting payload from client.

It can be

a) Query String

b) Form Data

c) Cookies etc.

FAQ: What is the role of Response Object on server?

Ans: It sends a response to client.

It can be

a) HTML

b) XML or JSON

c) File etc.

FAQ: What are various Status Codes?

Ans: A network application responds to client request using various status codes.

They are reserved for specific actions

2xx	Initial / confirmation / process
3xx	Methods / Actions
4xx	Client Side Issues
5xx	Server Side Issues

Note: You can send a response to client only when request is OK [200].

You can send other status messages to client on various issues.

writeHead():

- It is a response method of Node HTTP server.
- It identifies the request type, content type, payload and other details.
- It is responsible for making the browser understand about various contents set as response.
- It uses **MIME [Multipurpose Internet Mail Extension]** to define file type.

Syntax:

```
res.writeHead( statusCode, statusMessage, { headers } );
```

Ex:

```
res.writeHead( 200, ‘OK’, { ‘content-type’: ‘text/html’ });
```

Ex: HTML Response

index.js

```
const http = require("http");

const server = http.createServer((req, res)=>{
    res.writeHead(200, 'OK', { 'content-type':'text/html' });
    res.end('<h2>Welcome to Node Web Server</h2>');
});
```

```
server.listen(2020);

console.log('Server Started : http://127.0.0.1:2020&#39;);
```

Ex: JSON Response

```
const http = require("http");


const server = http.createServer((req, res)=>{

    const data = {

        Id: 1, Name: 'Mobile', Price: 45000, Cities: ['Delhi', 'Hyd']

    }

    res.writeHead(200, 'OK', { 'content-type':'application/json' });

    res.end(JSON.stringify(data));

});


server.listen(2020);

console.log('Server Started : http://127.0.0.1:2020&#39;);
```

Configuring Customized Response:

- You can identify the request URL and send a customized response.
- Request object URL returns the current virtual path.
- You can configure a condition to send different responses for various requests.

Syntax:

```
switch(req.url)

{
```

```
            case “/“:
                res.end( ‘default’ );
                break;
            case “/path”:
                res.end(“specific”);
                break:
        }
```

Ex:

```
const http = require("http");

const server = http.createServer((req, res)=>{
    res.writeHead(200, 'OK', { 'content-type': 'text/html' });
    switch(req.url){
        case "/":
         res.end('<h2>Home Page </h2>');
         break;
       case "/mobiles":
         res.end('<h2>Mobiles Home </h2><p>iPhone | Realme | Samsung</p>');
         break;
       default:
         res.end('<h2>Not Found</h2><p>Requested URL not found</p>');
         break;
    }
```

```
});


server.listen(2020);

console.log('Server Started : http://127.0.0.1:2020&#39;);
```

MongoDB

MongoDB Connection with Node JS

Day-5

Feb 11, 2026

Summary - Node Built-in Modules

- OS
- Path
- FS
- URL
- HTTP
- EVENTS

Node Connecting with Database

- A server side application connects with backend database by using **Data Provider.**
- Different databases require different providers.
- Various providers for database includes **OLEDB, ODBC, etc.**
- Node JS server app requires a suitable provider for connecting with different databases.
- Provider comprises of set of classes and functions responsible for handling

interaction with database.

Connecting with MongoDB:

- Node JS requires a **database driver** through provider to communicate with database.
- MongoDB has 2 drivers
    a) Mongoose
    b) MongoDB
- Mongoose is good for **ORM,** where logic is written for business requirements.
- MongoDB is good for applications where logic directly interacts with data.

1. Install MongoDB drivers for project

```
> npm install mongodb --save
```

2. Import **"MongoClient"** class from **mongoDB** module.

```
const  mongoClient = require("mongodb");
```

3. You can connect with MongoDB server using **connect()** method.

```
mongoClient.connect( "connection_string" )
            .then()
            .catch()
            .finally()
```

MongoDB Connection String : **mongodb://127.0.0.1:27017**

Ex: server.js

```
const mongoClient = require("mongodb").MongoClient;

mongoClient.connect('mongodb://127.0.0.1:27017').then(()=>{
   console.log(`Connected with MongoDB Server..`);
}).catch(error => {
   console.log(error);
})
```

4. “connect()” method is a promise that returns a **ClientObject** when connection is resolved.

Syntax:

```
mongoClient.connect( connection_string )
                .then( function(clientObject) {

                })
```

- Client Object can communicate with any database on server.

Syntax:

```
.then(function (clientObject) {
```

```
let database = clientObject.db("database_name");

})
```

- The database reference is responsible for connecting and interacting with various database objects like collections, views etc.

Syntax:

```
let database = clientObject.db("database_name");
database.collection("name").find()
                                .insertOne() etc..
```

Reading Data From Collection:

- Collection reference can use the methods
  a) find()
  b) findOne()
- **find()** requires **toArray()** to return all matching documents and convert to array.
- **toArray()** is a promise that returns array of documents when resolved.

Syntax:

```
database.collection("sampledb").find({ }).toArray().then( documents => {
    console.log(documents);
})
```

Ex: server.js

```
const mongoClient = require("mongodb").MongoClient;


mongoClient.connect('mongodb://127.0.0.1:27017').then((clientObj)=>{


    let database = clientObj.db('sampledb');

    database.collection('customers').find({}).toArray().then(documents=>{

        console.log(documents);

    })


}).catch(error => {

      console.log(error);

})
```

Ex: Web App on server connecting with Database

```
const http = require("http");

const mongoClient = require("mongodb").MongoClient;

const connectionString = "mongodb://127.0.0.1:27017";


const server = http.createServer((req, res)=>{



     if(req.url=="/"){
```

```
        res.writeHead(200, 'OK', { 'content-type': 'text/html' });

        res.end("<h2>Connected MongoDB Database</h2><p>You can request customers /
products / vendors</p>");



    } else if(req.url=="/customers"){



            res.writeHead(200, 'OK', { 'content-type': 'application/json' });

            mongoClient.connect(connectionString).then(clientObj=>{

            let database = clientObj.db("sampledb");

            database.collection("customers").find({}).toArray().then(documents=>{

                res.end(JSON.stringify(documents));

            })

        })



    }

})

server.listen(3030);

console.log(`Server Started : http://127.0.0.1:3030`);
```

Routes:

http://127.0.0.1:3030

http://127.0.0.1:3030/customers

Middleware Framework

Day-6

Feb 12, 2026

Node & MongoDB

- MongoDB Drivers
- Connecting with Database
- Reading Data from Collection

find() & findOne():

- find() returns an array of documents, even when the query is configure to return one.
- findOne() returns exactly one matching document.

find() => [ ]

findOne() => { }

Ex:

server.js

```
const mongoClient = require("mongodb").MongoClient;


mongoClient.connect('mongodb://127.0.0.1:27017').then(clientObj=>{
    var database = clientObj.db('sampledb');
    database.collection('products').findOne({id:1}).then(document=>{
        console.log(document);
    })
});
```

Inserting Document:

- You need configure data to insert on request.

- You can access the form data through middleware.

- It requires a request object that collects data from client.

- The mongoDB insertOne() & insertMany() are used to insert data.

Syntax:

```
var user = { key:value  }


database.collection('name').insertOne(user).then(()=>{}).catch(()=>{})
```

Update & Delete

- You can use the mongoDB commands

  a) updateOne()

  b) deleteOne()

  c) updateMany()

  d) deleteMany()

Syntax:

```
database.collection('name').updateOne( { find } , { $set: { } }).then(()=>{})


database.collection('name').deleteOne( { find } ).then(()=>{ } )
```

Syntax: Previous Versions with Callbacks

```
database.collection('name').updateOne( { find }, {set}, function(err, document){
        if(!err)
```

```
            {

            }

        })
```

Ex:

```
const mongoClient = require("mongodb").MongoClient;



mongoClient.connect('mongodb://127.0.0.1:27017').then(clientObj=>{

    var database = clientObj.db('sampledb');



    database.collection('customers').updateOne({username:'sanjay'}, {$set:{username:'Sanjay Kumar', age:27}}).then(()=>{

        console.log('Updated Customer details..');

    })

});
```

Server Side Framework

- Framework is a software architectural pattern.
- Real world software development uses
    a) Design Patterns
    b) Architectural Patterns

- Design Patterns are solutions for software design problems that you encounter while

building any application.

- There are 23 design patterns known as **GoF** Gang of Four patterns.

- They are categorized into 3 groups

  a) Creational Patterns

  b) Structural Patterns

  c) Behavioural Patterns

- Architectural Patterns are same like design patterns but have a broader scope, they can build and control the application flow.

- The popular architectural patterns

  a) MVC

  b) MVP

  c) MVVM

  d) Micro Services

  e) Blackboard System

  f) Broker Pattern

  g) Event Driven Architecture etc.

- Framework acts as a middleware for your application, which handles the communication between client, app and database.

- Node JS can use various middleware frameworks like

  a) Express JS

  b) Koa

  c) NestJS

  d) Fastify

  e) Hapi etc.

Express JS

- It is a middleware framework for Node JS.
- Middleware enables communication in network applications.
- It handles request from client, process request on server and sends a customized response for every request.

Express JS

Day-7

Feb 13, 2026

Express Server App:

1. Install Express for project

```
> npm install express --save
```

2. Import express library

```
const express = require("express");
```

3. Configure server application using express

```
const app = express();
```

4. Configure requests to handle

```
app.get( )
```

app.post() etc..

GET for fetching

POST for saving

PUT for updating

DELETE for removing etc.

5. Express uses **Route Middleware** to handle navigation in server application.

- Routing is a technique used in web application to enable communication and navigation.
- Express JS has a built-in Routing library that allows to configure routes for application.

Syntax:

```
app.get( “route_path”, function (req, res, next) {

})
```

6. Start application using “listen()” method

```
app.listen(port);
```

7. Every request for server comprises of 3 objects

a) request

b) response

c) next

- Request comprises of properties and methods used to access data

req.params

req.body

req.cookies

req.host etc.

- Response comprises of members to send a customized response

res.send()

res.json()

res.end() etc.

- Next is a middleware function that executes the next request in process.

Ex: server.js

```
const express = require("express");
const app = express();


app.get('/', (req, res, next)=>{
    res.send("<h1>Home</h1>");
    res.end();
    next();
});


app.get('/products', (re, res, next)=>{
```

```
    res.send([{Name:'TV'}, {Name:'Mobile'}]);
    res.end();
    next();
})

app.listen(4040);
console.log(`Server Started : http://127.0.0.1:4040`);
```

8. Express supports wild card routes, which allows to use a regular expression to handle request.

   Syntax:

```
app.get( /colou?r/ , (req, res, next) => {

})

app.get( '/col*r', () => { })
```

- Express router can respond to request that doesn't match with any of the given route paths. It uses a wild card character "*".

  Syntax:

```
app.get("*", (req, res, next)=>{
    // not-found
```

```
})
```

Note: The wild card "*" is deprecated in latest express routing.

It is configured alternatively using router module.

9. Route can be configured with parameters. The route parameters are used to query any content inside specific path.

Configure Parameter:

```
app.get("path/:param1/:param2..", (req, res, next)=>{ })
```

Parameters are defined in URL

http://server.com:port/path/value1/value2

Accessing parameters

```
req.params;            // It returns an object { } with params

{
    param1: value1,
    param2: value2
}
```

Ex:

```
const mongoClient = require("mongodb").MongoClient;

const express = require("express");


const connectionString = "mongodb://127.0.0.1:27017";


const app = express();


app.get('/', (req, res, next)=>{

    res.send("<h1>API Home</h1>");

    res.end();

    next();

})


app.get('/products', (req, res, next)=>{


     mongoClient.connect(connectionString).then(clientObj =>{

          var database = clientObj.db('sampledb');

          database.collection('products').find({}).toArray().then(documents=>{

               res.send(documents);

               res.end();

               next();

          });

     });
```

```
});

app.get('/product/:id', (req, res, next)=>{

    mongoClient.connect(connectionString).then(clientObj =>{
        var database = clientObj.db('sampledb');
        database.collection('products').findOne({id:parseInt(req.params.id)}).then(document=>{
            res.send(document);
            res.end();
            next();
        })
    });

});

app.listen(4040);
console.log('Server Started : http://localhost:4040&#39;);
```

Express App configuration

Day-8

Feb 14, 2026

- **Express Routes**

 - **Route Parameters**

 - **Wild Card Routes**

1. **Configuring app to use various status codes**

```
app.use( (req, res) =>{
        res.status(code).send('message | data | resource');
})
```

Ex: response if requested path is not found.

```
app.use( (req, res)=> {
        res.status(404).send('Path you requested not found');
})
```

2. Configuring app to use "body-parser"

- Body Parser is a library required to convert the form body into JSON.
- Express latest versions provide "body-parser" implicitly.
- You have to use body-parser to **encode** the URL data and convert into JSON.

Syntax:

```
app.use(express.urlencoded( {extended: true }));
app.use(express.json());
```

- It is mandatory to configure body parser in order handle actions like POST, PUT and DELETE.

3. Configuring app to handle CORS

- CORS is Cross Origin Resource Sharing

- It is a middleware required to enable or disable various requests in API.

- You can restrict GET, POST, PUT, DELETE according to the state and situation.

- CORS is an external library that your express API must use.

```
> npm install cors ;

const cors = require("cors");

app.use(cors({ cors_actions }));
```

- You can't POST, PUT or DELETE if CORS is not configured for API.

```
app.get('path',  cors(corsOptions), (req, res)=> {

})

corsOptions = { origin: 'url',  credentials: boolean, .. }
```

4. Configuring Static resources for app

- API can't access the static resources in application. - Static resources include

  a) images

  b) html

  c) css etc.

- Express provides a "static()" method that can configure the path for

  static resources. So that you can access the resources form specified path

directly.

Syntax:

```
app.use(express.static( 'public'));
```

- It is recommended to fetch path upto root level for production environment.

```
const path = require("path");
app.use(express.static(path.join(__dirname, 'public'));
app.use(express.static(path.join(__dirname, 'assets'));
```

- You can configure multiple folders for static.
- You can request static resources directly

http://127.0.0.1:4040/banner.jpg => keep banner.jpg in public folder

```
project_root_folder
|
|_ server.js
|
|_ public
|   |_index.html
|   |_banner.jpg
|
|_ package.json
```

5. Configuring View engine for express

- View is the UI generated by server.

- Server can send UI components generated dynamically on client request.

- It requires the support of various view engines like **PUG**.

API CRUD and PUG Engine

Day-9

Feb 16, 2026

Create API for handling CRUD operations:

1. Install all required libraries

```
> npm  install   express   cors    mongodb    --save
```

2. Create a new file “api.js”

```
const mongoClient = require("mongodb").MongoClient;

const cors = require("cors");

const express = require("express");


const connectionString = "mongodb://127.0.0.1:27017";

const app = express();


app.use(express.urlencoded({extended:true}));

app.use(express.json());

app.use(cors());
```

```
// Routes

app.post('/add-user', (req, res)=>{

    var user = {
        user_id : req.body.user_id,
        user_name: req.body.user_name,
        dob : new Date(req.body.dob),
        email: req.body.email
    }

    mongoClient.connect(connectionString).then(clientObj=>{
        var database = clientObj.db("project");
        database.collection("users").insertOne(user).then(()=>{
            console.log('Record Inserted');
            res.end();
        })
    })

});

app.put('/edit-user/:userid', (req, res)=>{

    var user = {
```

```
        user_id : req.body.user_id,
        user_name: req.body.user_name,
        dob : new Date(req.body.dob),
        email: req.body.email
    }

    mongoClient.connect(connectionString).then(clientObj =>{
        var database = clientObj.db("project");
        database.collection('users').updateOne({user_id:req.params.userid},{$set:user})
        .then(()=>{
            res.send(user);
            console.log('Updated');
            res.end();
        })
    })

});

app.delete('/delete-user/:userid', (req, res)=>{

    mongoClient.connect(connectionString).then(clientObj=>{
        var database = clientObj.db('project');
        database.collection('users').deleteOne({user_id:req.params.userid})
        .then(()=>{
            console.log('deleted..');
```

```
            res.end();
        })
    })
});

app.get('/users', (req, res)=>{

    mongoClient.connect(connectionString).then(clientObj=>{
        var database = clientObj.db('project');
        database.collection('users').find({}).toArray().then(documents=>{
            res.send(documents);
            res.end();
        })
    })
});

app.listen(4040);
console.log(`API Started : http://127.0.0.1:4040`);
```

3. Install API debugger
    - Postman
    - Fiddler
    - Swagger etc.

API Routes:

POST /add-user => Postman => POST => body => form-urlencoded

PUT /edit-user/user_id

DELETE /delete-user/user_id

GET /users

GET /users/:user_id

- Response object can use “redirect()” to navigate dynamically.
- It requires an existing route path on server.

Syntax:

```
res.redirect('/path');
```

Server Side Templates

- Server side application can configure UI dynamically.
- It allows to create a User Interface for client request and render to client dynamically.
- It requires a View Engine.
- **View Engines** are used in server side scripting technologies to create templates and render to client.
- Various server side scripting technologies used different view engines like **Spark, Nahml, Pug, Razor, ASPX,** etc.
- Node JS uses **PUG** as view engine.

1. Install PUG view engine

```
> npm install pug --save
```

2. Set pug as view engine for your application

```
app.set('view engine', 'pug')
```

3. Add view templates [View = UI]

```
index.pug
home.pug
about.pug
```

4. Build server side UI use template properties and methods

```
index.pug

doctype  html
html
    head
        title = "Some title"
    body
        h1   "some text"
        p    "some para"
```

5. Render on specific request

```
app.get(‘/path’, (req, res)=>{
        res.render(‘view_name’, { });      // it refers to PUG template file name
})
```

Ex: start.js

```
const express = require('express');
const path = require('path');
const cors = require('cors');

const app = express();
app.use(cors());

app.set('view engine', 'pug');
app.set('views', path.join(__dirname, 'views'));

app.get('/about', (req, res)=>{
   res.render('about');
})

app.listen(3030);
console.log('Server Started http://127.0.0.1:3030&#39;);
```

jQuery

Day-10

Feb 17, 2026

jQuery

- jQuery is a Javascript library.
- Library provides pre-defined functions to handle various interactions.
- You can **write less and do more**.
- jQuery can handle
  a) DOM Interactions
  b) BOM Interactions
  c) AJAX
  d) FORM Validations
  e) CSS Animations etc.
- jQuery built by **John Resign** in early 2006.
- It reduces the compatibility issues.

Setup jQuery for project:

1. Install jQuery library

```
> npm install jquery --save
```

2. Link jQuery library file to HTML page

```
<script src="../node_modules/jquery/dist/jquery.js"> </script>
```

3. jQuery library is defined in an anonymous function, which is invoked with "$" reference.

- Library is always passive.
- It is not active.
- You have to make it active explicitly.

```
<script>
    $( function(){

            // jQuery functions are accessible only inside this scope.

    })
</script>
```

jQuery Reference Technique:

- It can use all CSS selectors to select or refer elements in page.
- CSS selectors include
    a) Primary
    b) Relational
    c) Attribute
    d) Dynamic Classes etc..

Syntax:

```
<img>
<button id="btnSubmit">
<div class="container">
```

$(“img”)

$(“#btnSubmit”)

$(“.container”)

$(“nav h2”)

$(“header > span”)

jQuery DOM methods & properties:

text() innerText / textContent

html() innerHTML

val() value

attr() attribute

prop() property

class() className

css() css styles

append()] as suffix

appendTo()]

prepend()] as prefix

prependTo()]

before()] above

after()] below

etc..

Ex:

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script src="../node_modules/jquery/dist/jquery.js"></script>
    <script>
        $(()=>{
            $("h1").text('jQuery');
            $("#msg").html('<b>Write Less, Do More</b>');
            $(".container").text('jQuery is a library for building UI');
            $("#txtName").val('John');
            $("img").attr("src","./banner.jpg");
        })
    </script>
</head>
<body>
    <img>
    <h1></h1>
    <p id="msg"></p>
    <div class="container"></div>
    <br><br>
    <div>
        User Name : <input type="text" id="txtName">
    </div>
</body>
```

```
</html>
```

Ex:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Document</title>

    <script src="../node_modules/jquery/dist/jquery.js"></script>

    <script>

        $(()=>{

            var categories = ['All', 'Electronics', 'Fashion', 'Footwear'];



            categories.map(category=>{

                $(`<li>${category}</li>`).appendTo('ol');

                $(`<option value=${category}>${category}</option>`).appendTo('select');

            })



            fetch('https://fakestoreapi.com/products&#39;)

            .then(response=> response.json())

            .then(products=>{

                products.map(product=>{

                    $(`<tr> <td>${product.title}</td> <td> <img src=${product.image} width="100"
height="100"> </td> </tr>`).appendTo("tbody");

                })
```

```
            })

            $("h1").before("Hi").after("Hello").prepend("left-").append("-right");
        })
    </script>
</head>
<body>
    <h1>Welcome</h1>
   <ol>

   </ol>
   <select>

   </select>
   <table width="100%" border="1">
      <thead>
        <tr>
            <th width="200"> Title</th>
            <th>Preview</th>
        </tr>
      </thead>
      <tbody>

      </tbody>
   </table>
```

```
</body>

</html>
```

jQuery Events:

- It uses all Javascript browser events.
- It completely depends on event listeners.
- However you can bind handler explicitly.

Syntax:

```
<button>

<select>


$(“button”).click( (e)=>{ })


$(“select”).change((e)=>{

    e.target.value;

    e.clientX;

})
```

Ex:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>Document</title>
    <script src="../node_modules/jquery/dist/jquery.js"></script>
    <script>
        $(()=>{
            $("button").click(()=>{
                var username = $("#txtName").val();
                $("h1").html(`Hello ! ${username} from ${$("select").val()}`);
            })
            $("select").change(()=>{
                var username = $("#txtName").val();
                $("h1").html(`Hello ! ${username} from ${$("select").val()}`);
            })
        })
    </script>
</head>
<body>
    <dl>
        <dt>User Name</dt>
        <dd><input type="text" id="txtName"></dd>
        <dt>Your City</dt>
        <dd>
            <select>
                <option>Select City</option>
                <option>Delhi</option>
                <option>Hyderabad</option>
```

```
            </select>
        </dd>
    </dl>
    <button>Submit</button>
    <h1></h1>
</body>
</html>
```

jQuery AJAX:

jQuery AJAX and UI

Day-11

Feb 18, 2026

- jQuery DOM

- jQuery Events

jQuery AJAX

- It is async.
- It returns data in the native format of data source.
- Explicit conversions not required.
- It returns status & status code with headers.
- It is good in error handling.
- It is better in security like CORS & XSRF.
- It provides a complete set of life cycle methods, which can track issues at every phase.
- jQuery AJAX methods include the following

```
.getJSON()
.ajax()
.ajaxStart()
.ajaxStop()
.ajaxComplete()
.ajaxSuccess() etc..
```

Syntax:

```
$.ajax( {
    method : “get | post | put | delete..”,
    url: “path”,
    data: “data_to_send_on_post”,
    success: (data)=> { },
    error: (err) => { }
} )
```

- AJAX life cycle phases include

  Start => Success => Complete => Stop => Error

- Error executes after every phase.

Ex:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Document</title>

    <script src="../node_modules/jquery/dist/jquery.js"></script>

    <script>

        $(()=>{

            $("button").click(()=>{

                 $.ajax({

                method: "get",

                url: "https://fakestoreapi.com/products/categories&quot;,

                success: (categories)=>{

                    categories.unshift('all');

                    categories.map(category=>{

                        $(`<option
value=${category}>${category.toUpperCase()}</option>`).appendTo("select");

                    })

                },

                error: (ex)=>{

                    console.log(ex);

                }



             })

            })
```

```
        })
        .ajaxStart(()=>{
            alert('Ajax request started..');
        })
        .ajaxComplete(()=>{
            alert('Ajax request completed..');
        })
        .ajaxSuccess(()=>{
            alert('Ajax request success in execution');
        })
        .ajaxStop(()=>{
            alert('Response ready to render');
        })
        .ajaxError(()=>{
            alert('Something went wrong..');
        })
    </script>
</head>
<body>
    <button>Load</button>
    <select>


    </select>
</body>
</html>
```

jQuery Animations

slide()

slideToggle()

fade()

fadeToggle()

show()

hide()

toggle() etc.

Syntax:

```
$("img").fadeToggle(interval);

$("#pic").toggle(2000);
```

jQuery UI

- It is similar to bootstrap.
- It has a UI library with

  a) Effects

  b) Components / Widgets

  c) Plugins etc.

1. Download jQuery UI library ZIP folder

   https://jqueryui.com/

2. Extract the ZIP folder

3. Copy all files

4. Go to project & add a new folder “jquery-ui”

5. Paste copied library files

6. Link the following files to your web page

```
<script src="../node_modules/jquery/dist/jquery.js"></script>
<script src="../jquery-ui/jquery-ui.js"></script>
<link rel="stylesheet" href="../jquery-ui/jquery-ui.css">
<link rel="stylesheet" href="../jquery-ui/jquery-ui.theme.css">
```

Interactions

draggable()

resizable()

sortable()

selectable() etc..

Syntax:

```
$(“img”).draggable()
$(“ul”).sortable()
```

Ex:

```
<!DOCTYPE html>
```

```html
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link rel="stylesheet" href="../jquery-ui/jquery-ui.css">
    <link rel="stylesheet" href="../jquery-ui/jquery-ui.theme.css">
    <script src="../node_modules/jquery/dist/jquery.js"></script>
    <script src="../jquery-ui/jquery-ui.js"></script>
    <script>
        $(()=>{
            $("figure").draggable();
            $("ol").sortable();
        })
    </script>
</head>
<body>
   <ol>
     <li>HTML</li>
     <li>Bootstrap</li>
     <li>JavaScript</li>
     <li>CSS</li>
     <li>SASS</li>
   </ol>
   <figure style="border:1px solid black;padding: 20px; width: 320px;">
```

```
    <img src="./banner.jpg" width="300">
    <figcaption>Ads</figcaption>
   </figure>
</body>
</html>
```

Widgets [Components]

accordion()

autocomplete()

dialog()

datepicker()

spinner()

tabs()

tooltip() etc..

Syntax:

```
$("#faqs").accordion()
$("div").dialog()
$("#date").datepicker()
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link rel="stylesheet" href="../jquery-ui/jquery-ui.css">
    <link rel="stylesheet" href="../jquery-ui/jquery-ui.theme.css">
    <script src="../node_modules/jquery/dist/jquery.js"></script>
    <script src="../jquery-ui/jquery-ui.js"></script>
    <script>
        $(()=>{
            $("#faqs").accordion();
            $("#msg").dialog();
            $("#date").datepicker();
        })
    </script>
</head>
<body>
    <input id="date">
    <div id="msg" title="Registered">
        <p>Registered successfully..</p>
    </div>
    <h1 align="center">Frequently Asked Questions</h1>
    <div id="faqs">
        <h3>What is Netflix?</h3>
        <div>
```

```
        <p>Lorem ipsum, dolor sit amet consectetur adipisicing elit. Esse modi quos, dolorum neque quisquam a, voluptas nostrum quaerat nobis facilis aliquam impedit molestias debitis ex sapiente quod recusandae iste nemo? Lorem ipsum dolor sit amet, consectetur adipisicing elit. Amet nesciunt ex sequi omnis quam! Ipsam ea molestiae nesciunt inventore repudiandae. Non quaerat autem sequi obcaecati odio, deleniti at assumenda molestias.</p>

      </div>

      <h3>How much doesn Netflix cost?</h3>

      <div>

        <p>Lorem ipsum, dolor sit amet consectetur adipisicing elit. Ipsa itaque sed, eos, quas quo ex repudiandae ratione neque cumque qui in ipsam quam expedita magnam temporibus optio consequuntur amet exercitationem. Lorem ipsum dolor sit amet consectetur, adipisicing elit. Rem commodi earum eveniet pariatur obcaecati iste, architecto alias a corporis. Repellat debitis consequuntur dolor illum soluta itaque facere consequatur modi libero. </p>

        <p>Lorem ipsum, dolor sit amet consectetur adipisicing elit. Ipsa itaque sed, eos, quas quo ex repudiandae ratione neque cumque qui in ipsam quam expedita magnam temporibus optio consequuntur amet exercitationem. Lorem ipsum dolor sit amet consectetur, adipisicing elit. Rem commodi earum eveniet pariatur obcaecati iste, architecto alias a corporis. Repellat debitis consequuntur dolor illum soluta itaque facere consequatur modi libero. </p>

      </div>

    </div>

</body>

</html>
```

jQuery Plugins

- Plugin is a set of pre-defined logic with design and functionality to handle various interactions.
- jQuery supports various 3rd party plugins
    a) jQuery Cookies
    b) jQuery Grid
    c) jQuery Dashboard
    d) jQuery Calendar etc.

Ex: Barcode

```
>npm install  jquery-ean13
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link rel="stylesheet" href="../jquery-ui/jquery-ui.css">
    <link rel="stylesheet" href="../jquery-ui/jquery-ui.theme.css">
    <script src="../node_modules/jquery/dist/jquery.js"></script>
    <script src="../jquery-ui/jquery-ui.js"></script>
    <script src="../node_modules/jquery-ean13/dist/jquery-ean13.min.js"></script>
    <script>
        $(()=>{
            $("#code").EAN13("9988812331111")
        })
    </script>
</head>
<body>
    <img src="./iphone-marine.png" width="300" height="300">
    <br><br>
```

```
    <canvas width="200" height="50" id="code"></canvas>

</body>

</html>
```

FullStack ToDo Application

Day-12

Feb 12, 2026

To-do Application

- User can register
- User can login
- User dashboard shows list of appointments.
- User can
  a) Add Appointment
  b) Edit Appointment
  c) Delete Appointment
  d) Search / Filter Appointments

Database: MongoDB

1. users [collection]
   - user_id
   - user_name
   - password
   - email
2. appointments [collection]
   - appointment_id
   - title

- description

- date

- user_id [FK - users]

API : [Node & Express]

Routes: [End Points]

GET	/users	get all users
GET	/appointments/user_id	get all appointments of specific user
GET	/appointment/id	get specific appointment by id
POST	/register-user	adds new user
POST	/add-appointment	adds new appointment
PUT	/edit-appointment/id	edits specific appointment
DELETE	/delete-appointment/id	deletes specific appointment

Setup Project 1. Create a new folder for project

"fullstack-todo-app"

2. Run the commands

```
>npm init -y
> npm install  express  cors  mongodb  jquery --save
```

3. Add public folder with "index.html"

4. Add server.js in root level

```
Fullstack-todo-app
|
|_ server.js
|_ public
|    |_index.html
|_package.json
|_node_modules
```

5. Install bootstrap as UI library

```
> npm install bootstrap  bootstrap-icons --save
```

jQuery Events for Embedded Elements

Day-13

Feb 20, 2026

jQuery Events for Embedded Elements:

- jQuery can't access elements directly to configure event from a multilevel hierarchy of UI components.
- It requires to map the element in document by using **"on()"** and configure event.

Syntax:

```
$(document).on("event", "element_reference", (e)=>{
    // e uses event args.
})
```